Redes neuronales para el reconocimiento de audio a partir de espectrogramas

Luis Felipe Calderón Pérez
Isaac David Brenes Torres
Juan Pablo Rodríguez Cano
Curso de Inteligencia Artificial
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica

***Resumen*—Este trabajo presenta la aplicación de redes neuronales convolucionales (CNN) para la clasificación multiclase de sonidos a partir de espectrogramas de audio. Se utiliza el conjunto de datos ESC-50, compuesto por grabaciones etiquetadas en diversas categorías sonoras. Se desarrollan dos modelos: una versión adaptada de LeNet-5 y una arquitectura alternativa basada en referencias de la literatura reciente. Ambos se entrenan con datos originales y con versiones aumentadas mediante técnicas inspiradas en audio, con el fin de evaluar su efecto sobre la generalización del modelo. El proceso de entrenamiento y validación se gestiona mediante PyTorch y la plataforma Weights Biases (WandB), que permite el seguimiento de métricas de desempeño. Los resultados obtenidos permiten comparar la eficiencia y precisión de ambas arquitecturas, aportando una perspectiva sobre el uso de CNN en tareas de reconocimiento de audio ambiental.**

Palabras clave — redes neuronales convolucionales, clasificación de audio, espectrogramas, aprendizaje supervisado.

I. Introducción

El reconocimiento automático de sonidos es un área de creciente interés dentro del campo del Deep Learning, con aplicaciones que van desde la seguridad acústica hasta los asistentes inteligentes y la monitorización ambiental. A diferencia del reconocimiento de voz tradicional, la clasificación de sonidos ambientales plantea desafíos adicionales debido a la alta variabilidad de las fuentes sonoras, los niveles de ruido y las condiciones de grabación.

En este contexto, las redes neuronales convolucionales (CNN) se han consolidado como una herramienta eficaz para la extracción y análisis de patrones espectrales. Al transformar los audios en espectrogramas, es posible reinterpretar las señales de audio como imágenes, lo que permite aprovechar las ventajas de las CNN en el procesamiento visual. Este enfoque facilita la detección de características temporales y frecuenciales relevantes sin requerir ingeniería manual de atributos.

El presente proyecto tiene como propósito aplicar arquitecturas convolucionales al problema de clasificación multiclase de sonidos, utilizando el conjunto de datos ESC-50, que contiene grabaciones de corta duración clasificadas en cincuenta categorías. Se desarrollan dos modelos: una versión adaptada de LeNet-5 y una arquitectura alternativa basada en la literatura reciente, con el fin de comparar su desempeño y capacidad de generalización.

Además, se implementan técnicas de data augmentation inspiradas en audio para evaluar su impacto en la robustez de los modelos ante la variabilidad del conjunto de datos. El entrenamiento y la evaluación se realizan en el entorno PyTorch, con registro y monitoreo de métricas a través de la plataforma Weights & Biases (WandB), garantizando la reproducibilidad y el análisis comparativo.

II. Metodología

II-A. Conjunto de Datos

Para el desarrollo de este trabajo se utilizó el conjunto de datos ESC-50 (Environmental Sound Classification Dataset), propuesto por Piczak (2015) [1]. Este dataset es ampliamente utilizado en tareas de clasificación de audio y constituye un estándar de referencia en el área de reconocimiento de sonidos ambientales.

El conjunto contiene 2000 grabaciones de audio con una duración de 5 segundos cada una, distribuidas en 50 clases balanceadas, con 40 ejemplos por categoría. Las clases incluyen sonidos de animales, entornos naturales, maquinaria, sonidos humanos no verbales e instrumentos musicales. Cada clip se encuentra almacenado en formato .wav con una frecuencia de muestreo de 44.1 kHz, codificado en mono y etiquetado según su categoría.

Los datos fueron obtenidos directamente desde el repositorio público del autor, disponible en GitHub.

II-B. Análisis Exploratorio de Datos

El análisis exploratorio inicial se centró en la distribución de clases, la duración y las características espectrales de los audios. Se verificó que las 50 categorías presentan un equilibrio uniforme en número de muestras, sin desbalance significativo entre clases. Se inspeccionaron aleatoriamente varias grabaciones para asegurar la integridad de los archivos y la correcta correspondencia entre nombre y etiqueta.

A nivel acústico, se observó que las grabaciones presentan variabilidad significativa en intensidad y rango de frecuencias, reflejando la diversidad de los entornos sonoros. Se identificó además que ciertas clases (por ejemplo, sonidos de motores o lluvia) presentan componentes frecuenciales continuas, mientras que otras (como ladridos o golpes) son de naturaleza impulsiva. Este comportamiento sugiere la pertinencia del uso

de representaciones espectrales como entrada para modelos convolucionales.

II-C. Tratamiento y preparación de los datos

En cuanto al conjunto de datos se realizó una validación completa de las rutas y estructuras del conjunto de datos, asegurando que todas las imágenes existieran físicamente en disco y que pudieran abrirse correctamente. Únicamente se conservaron aquellas con modo de color `RGBA`, garantizando así la consistencia en el número de canales de entrada del modelo. A partir del archivo `CSV` original, se generaron las rutas absolutas correspondientes y se verificó la integridad de cada registro.

Posteriormente, se analizó la distribución de clases dentro del conjunto de datos y se observó un desbalance significativo entre categorías. Para mitigar este efecto, se aplicó un muestreo estratificado basado en la clase con menor cantidad de muestras, equilibrando así el número de imágenes por categoría. Además, se calculó el brillo promedio de cada imagen utilizando los tres canales `RGB`, lo que permitió visualizar la distribución lumínica mediante histogramas comparativos antes y después del balanceo. Este análisis ayudó a identificar valores atípicos de brillo, los cuales se trataron mediante *Winsorization*, ajustando los valores por encima del percentil 95 y por debajo del percentil 5 para reducir la influencia de extremos sin eliminar observaciones.

Con el objetivo de facilitar el procesamiento posterior, las etiquetas categóricas se transformaron en valores numéricos mediante el uso de `LabelEncoder` de *scikit-learn*, lo cual permitió una integración directa con las funciones de pérdida del modelo. Una vez codificadas, se procedió a dividir el conjunto de datos en subconjuntos de entrenamiento, validación y prueba mediante una estrategia de *stratified shuffle split*, que consideró tanto la clase como el brillo promedio de las imágenes. De esta forma, se garantizó una representación equilibrada de las distintas clases en cada subconjunto, manteniendo una proporción de 70 % para entrenamiento, 15 % para validación y 15 % para prueba.

Adicionalmente, se incorporó un conjunto aumentado de imágenes mencionado en el subapartado anterior y se fusionó con el conjuto de datos crudos, con el propósito de incrementar la diversidad de patrones visuales y mejorar la capacidad de generalización del modelo. Tanto el conjunto original como el aumentado fueron procesados bajo el mismo flujo de validación, balanceo y división.

Finalmente, todas las imágenes fueron redimensionadas a 224×224 píxeles y normalizadas en un rango de $[-1, 1]$, utilizando una media y desviación estándar de 0.5 por canal. Este preprocesamiento aseguró una escala uniforme entre las muestras, evitando sesgos por diferencias de iluminación o color. Posteriormente, se construyeron `DataLoaders` con un tamaño de lote de 32 y barajado aleatorio de las muestras, optimizando la eficiencia del entrenamiento y reduciendo posibles dependencias entre lotes consecutivos.

II-D. Data Augmentation en espectrogramas

Para mejorar la capacidad de generalización de los modelos y aumentar la diversidad del conjunto de datos, se implementó una estrategia de aumento de datos (data augmentation) directamente sobre las señales de audio antes de su transformación en espectrogramas. Este enfoque permite crear variaciones realistas de las muestras originales sin perder la esencia de la información sonora, lo cual es crucial para evitar el sobreajuste en modelos de aprendizaje profundo con conjuntos de datos limitados como ESC-50.

El procedimiento consistió en aplicar varias técnicas de augmentación sobre los audios originales. Las transformaciones seleccionadas fueron:

- **AddGaussianNoise**: Se añadió ruido gaussiano con una amplitud variable, simulando condiciones acústicas más ruidosas o interferencias comunes en grabaciones reales.
- **TimeStretch**: Se modificó la velocidad de reproducción del audio sin alterar su tono, con factores entre 0.8 y 1.25, para representar diferentes velocidades de eventos sonoros.
- **PitchShift**: Se desplazó el tono del audio hasta 2 semitonos arriba o abajo, emulando variaciones en la fuente sonora o efectos de grabación.
- **Shift**: Se desplazó la señal temporalmente (fracción del total), generando pequeños desfases que aumentan la variabilidad temporal.

Estas transformaciones se aplicaron de manera probabilística y combinada a cada archivo de audio, generando una versión aumentada para cada muestra original. Posteriormente, tanto la señal cruda como la aumentada se convirtieron en espectrogramas.

La elección de aplicar data augmentation sobre el audio y no directamente sobre las imágenes espectrales responde a que las transformaciones acústicas generan variaciones más naturales y significativas, preservando la coherencia temporal y frecuencial inherente a las señales sonoras. Esto es especialmente importante para evitar crear muestras irreales o distorsionadas que podrían confundir al modelo durante el entrenamiento.

III. Implementación de modelos

Se implementaron 3 Modelos:

III-A. Modelo A clásico

La implementación de la arquitectura LeNet-5 se basó en las propuestas de E. Lewinson [2] y Nouman [3]. Se adaptó para que la entrada de las imágenes tuviera un tamaño de 224×224 píxeles y 4 canales (RGBA). En consecuencia, se recalculó el tamaño de la salida que alimenta la capa fully connected, obteniéndose un valor de 288,120.

Se realizó una exploración de hiperparámetros mediante Weights & Biases (wandb), ajustando la tasa de aprendizaje, el tamaño de lote y el número de épocas. Cada configuración se evaluó utilizando las métricas accuracy, precision, recall, F1-score y AUC en los conjuntos de validación y prueba, lo que permitió identificar el mejor modelo tanto con los datos originales como con los datos aumentados.

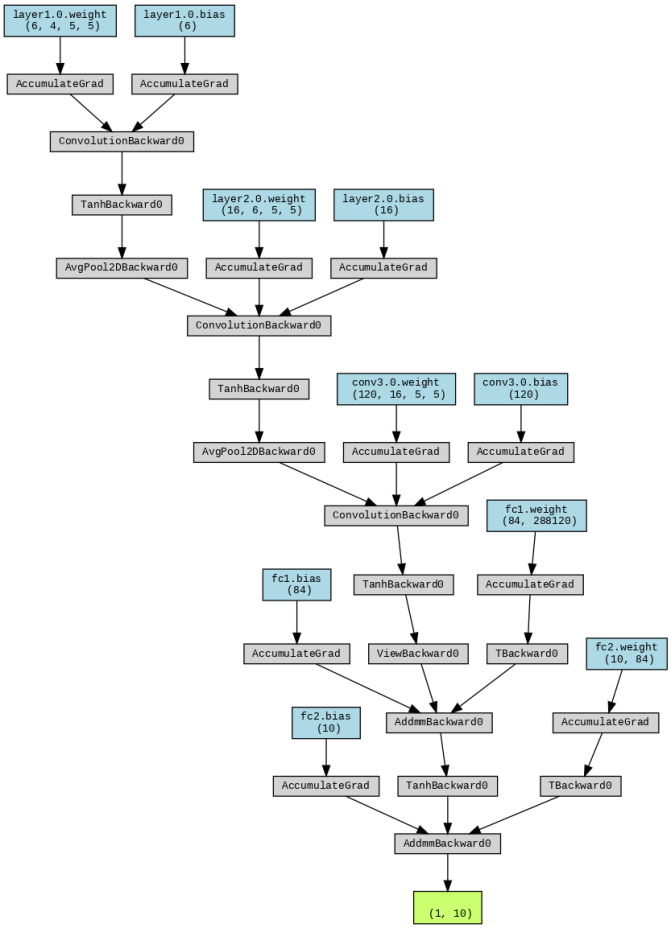

Figura 1. Arquitectura LeNet5 clásica

Para mejorar la generalización, se incorporó una regularización L2.

La arquitectura se puede observar en 1

III-B. Modelo A con dropout

A partir de los resultados obtenidos con el Modelo A clásico, se decidió incorporar una función de dropout en la capa fully connected con el fin de mejorar la capacidad de generalización del modelo.

Al igual que en el modelo anterior, se llevó a cabo una exploración de hiperparámetros mediante wandb (tasa de aprendizaje, tamaño de lote y número de épocas), evaluando cada configuración con las métricas accuracy, precision, recall, F1-score y AUC en validación y prueba, lo que permitió obtener el mejor desempeño con los datos crudos y aumentados.

La arquitectura se puede observar en 2

III-C. Modelo B

El modelo que se escogió para este proyecto se basa en los estudios hechos por Friedrich Wolf-Monheim [2], donde se entrena una arquitectura que se define por las siguientes capas para poder clasificar el conjunto de datos ESC-50

1. Capa de normalización por lotes para normalizar los espectrogramas en el eje de frecuencia

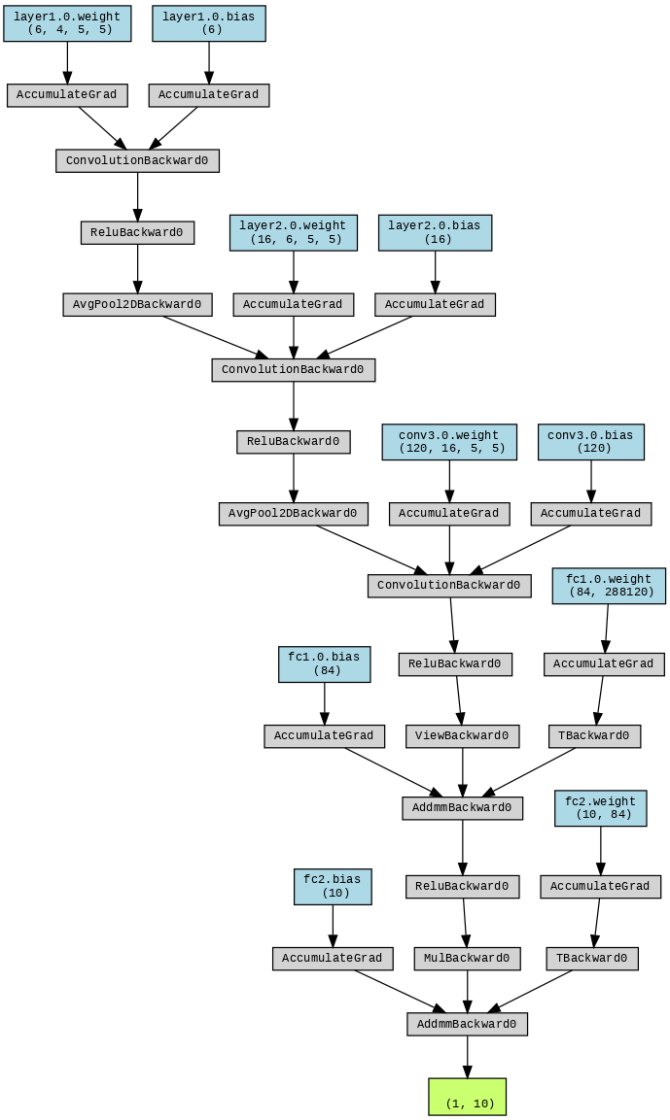

Figura 2. Arquitectura LeNet5 con dropout

2. Capa de convolución 2D con 64 filtros de 3 píxeles de altura, 3 píxeles de ancho, función de activación de unidad lineal rectificada (ReLU) y relleno "same"
3. Capa de agrupamiento máximo para datos espaciales 2D con una ventana de agrupamiento de 2x2
4. Capa de convolución 2D con 128 filtros de 3 píxeles de altura, 3 píxeles de ancho, función de activación de unidad lineal rectificada (ReLU) y relleno "same"
5. Capa de agrupamiento máximo para datos espaciales 2D con una ventana de agrupamiento de 2x2
6. Capa de convolución 2D con 256 filtros de 3 píxeles de altura, 3 píxeles de ancho, función de activación de unidad lineal rectificada (ReLU) y relleno "same"
7. Capa de agrupamiento máximo para datos espaciales 2D con una ventana de agrupamiento de 2x2
8. Capa de convolución 2D con 256 filtros de 3 píxeles de altura, 3 píxeles de ancho, función de activación de

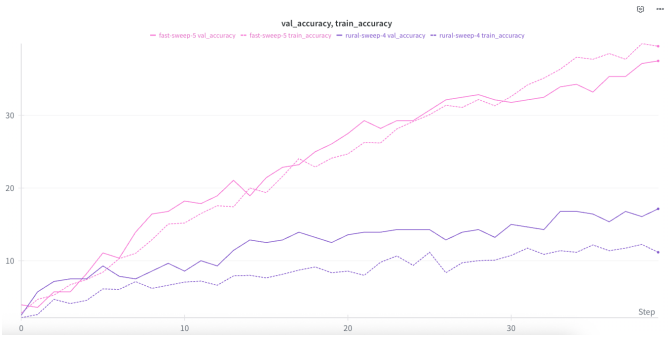

Figura 3. Exactitud de los mejores modelos entrenados en el sweep de la arquitectura A con Dropout con datos crudos

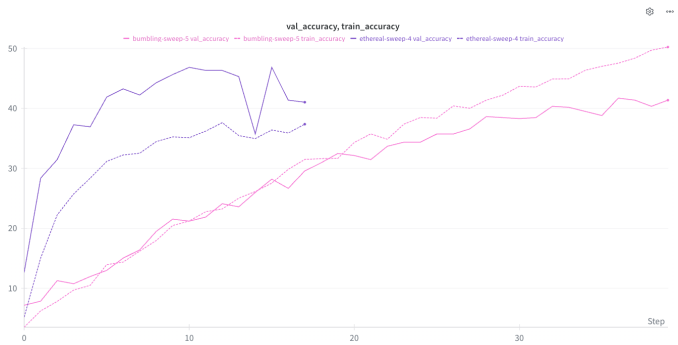

Figura 4. Exactitud de los mejores modelos entrenados en el sweep de la arquitectura A con Dropout con datos aumentados

unidad lineal rectificada (ReLU) y relleno "same"

9. Capa de agrupamiento máximo para datos espaciales 2D con una ventana de agrupamiento de 2x2
10. Capa de aplanamiento
11. Capa de red neuronal densamente conectada con 256 unidades y función de activación de unidad lineal rectificada (ReLU)
12. Capa de abandono con una tasa de 0.5
13. Capa de red neuronal densamente conectada con 50 unidades y función de activación softmax para convertir el vector de salida de valores a una distribución de probabilidad

Se puede notar que el método de Wolf-Monheim aplica una normalización al eje de frecuencia, lo cual no es lo mismo que aplicar normalización para una imagen en ese eje. Sin embargo, es conveniente que los espectrogramas muestra la amplitud de la frecuencia con la intensidad de pixeles para una frecuencia dada, por lo que se opta por normalizar los pixeles en ese eje de la imagen, ya que sería un proceso similar.

IV. Resultados Experimentales

A continuación se presentan las gráficas de los mejores 2 modelos que se encontraron haciendo un sweep de 5 modelos para cada arquitectura, es decir, cada arquitectura se entrenó con 5 modelos con combinaciones de hiperparámetros diferentes escogidos según una lista constante para todos los modelos. Los valores de esas listas son los siguientes:

- Tasa de aprendizaje: 0.0001, 0.0005, 0.001
- Dropout: 0.3, 0.5 y 0.7
- Optimizador: Adam y SGD

IV-A. Comparación de Métricas en Validación

A continuación, se muestran las metricas obtenidas para los 3 modelos entrenados con datos crudos y aumentados

Se puede notar que las métricas del modelo A con dropout con datos crudos no tiene datos, esto sucede porque a pesar de que el sweep que se muestra en la figura 3 sí obtuvo resutlados de exactitud de 39 % y 37 % para el conjunto de entrenamiento y validación, respectivamente, estos no se pudieron replicar por razones desconocidas.

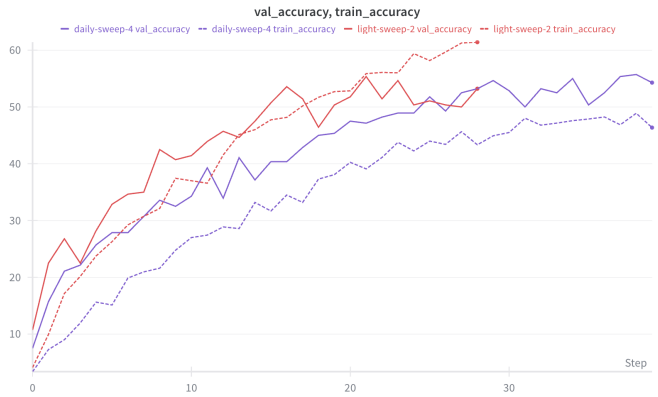

Figura 5. Exactitud de los mejores modelos entrenados en el sweep de la arquitectura B y datos crudos

Modelo	Dataset	Exactitud	Precisión	Recall	F1
A	crudo	0.36	34	218.0.36	0.31
A	aumentado	0.33	0.32	0.33	0.28
A (Drop)	crudo	n/a	n/a	n/a	n/a
A (Drop)	aumentado	0.89	0.90	0.88	0.88
B	crudo	0.53	0.55	0.53	0.51
B	aumentado	0.54	0.56	0.55	0.53

Cuadro I
Resultados de evaluación de modelos con conjunto de prueba

V. Análisis de Resultados

Los resultados obtenidos en este estudio ofrecen una visión clara sobre el desempeño de las arquitecturas CNN aplicadas para la clasificación de sonidos ambientales a partir de espectrogramas, así como el impacto del aumento de datos y la regularización en la capacidad de generalización.

V-A. Comparación entre Modelos

El Modelo A clásico, basado en una adaptación de LeNet-5, mostró un desempeño limitado cuando se entrenó únicamente con datos crudos, alcanzando una exactitud cercana al 36

La incorporación de técnicas de aumento de datos mejoró levemente el rendimiento del Modelo A clásico, aunque se evidenció un riesgo de sobreajuste debido a la diferencia significativa entre las métricas de entrenamiento y validación.

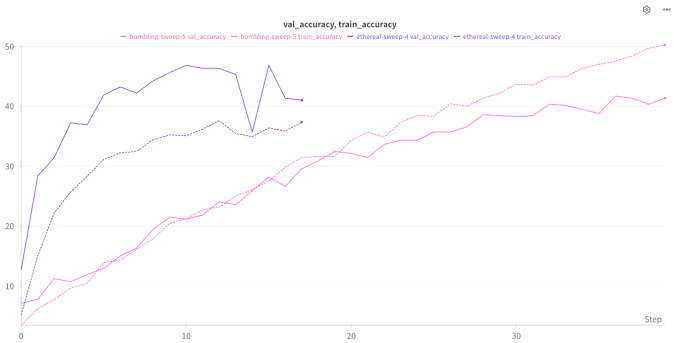

Figura 6. Exactitud de los mejores modelos entrenados en el sweep de la arquitectura B y datos aumentados

Esto motivó la introducción del dropout en la arquitectura, generando el Modelo A con dropout, que consiguió mejorar sustancialmente la generalización, alcanzando una exactitud del 89

El Modelo B, basado en una arquitectura más profunda y compleja inspirada en trabajos recientes, logró un desempeño intermedio con accuracies en torno al 53-54

V-B. Impacto del Data Augmentation

El uso de data augmentation tuvo un impacto positivo en todos los modelos evaluados. El aumento de la diversidad de patrones visuales en los espectrogramas permitió que los modelos aprendieran características más generales y menos específicas a las muestras originales, lo que se traduce en mejor desempeño en los conjuntos de validación y prueba. En particular, el Modelo A con dropout evidenció la mayor mejora, reflejando la sinergia entre técnicas de regularización y aumento de datos para combatir el sobreajuste.

V-C. Generalización y Robustez

El análisis de las curvas de pérdida y métricas durante el entrenamiento muestra que las pérdidas de entrenamiento y validación se mantuvieron relativamente cercanas en los modelos con técnicas de regularización y data augmentation, lo que indica buena capacidad de generalización. Sin embargo, la sensibilidad a hiperparámetros como la tasa de aprendizaje destaca la importancia de realizar una exploración cuidadosa y ajustes finos para obtener resultados óptimos.

V-D. Limitaciones Observadas

A pesar de las mejoras obtenidas, existen limitaciones que deben considerarse. Primero, el tamaño total del dataset (2000 muestras) es relativamente pequeño para entrenar arquitecturas profundas de manera efectiva, lo que puede limitar la capacidad de aprendizaje de patrones complejos y la robustez del modelo. Además, la transformación de audios a espectrogramas con un redimensionamiento a 224×224 píxeles puede haber provocado pérdida de información relevante, afectando la precisión.

Otra limitación está relacionada con la distribución balanceada pero con baja cantidad de ejemplos por clase, lo que aumenta la dificultad para aprender características discriminativas específicas para cada categoría sonora.

V-E. Interpretabilidad y Aplicaciones

La arquitectura sencilla del Modelo A facilita la interpretación de los pesos y filtros aprendidos, lo cual es valioso para entender qué patrones espectrales contribuyen a la clasificación. En contraste, el Modelo B, aunque más complejo, ofrece mayor potencial para capturar relaciones no lineales y jerárquicas entre características espectrales, lo que puede ser útil para tareas de reconocimiento más exigentes.

VI. Conclusiones

En este trabajo se exploró la aplicación de redes neuronales convolucionales para la clasificación multiclase de sonidos ambientales a partir de espectrogramas, utilizando el dataset ESC-50. Se implementaron y compararon tres modelos principales: una adaptación clásica de LeNet-5, una versión mejorada con dropout y una arquitectura más profunda basada en trabajos recientes.

Los resultados muestran que el modelo clásico de LeNet, entrenado con datos crudos, presentó una precisión limitada, con un exactitud de prueba de 36

Al aplicar técnicas de aumento de datos, el mismo modelo clásico mostró un desempeño similar, con una exactitud de prueba de 33

Para mitigar este problema, se introdujo una capa de dropout en el modelo, la cual mejoró notablemente el desempeño al entrenar con datos aumentados. El Modelo A con dropout alcanzó una exactitud de 89

Por otro lado, el Modelo B, con una arquitectura más compleja, logró resultados intermedios, con un exactitud de prueba cercano al 53-54

Se concluye que la efectividad del reconocimiento de audio a partir de espectrogramas con CNN depende en gran medida de la arquitectura utilizada, la cantidad y diversidad del conjunto de datos, y las técnicas de regularización y aumento empleadas. La inclusión de dropout junto con data augmentation fue clave para alcanzar los mejores resultados en este estudio.

Referencias

[1] The pandas development team, “User Guide — pandas.” [Online]. Available: https://pandas.pydata.org/docs/user_guide/index.html.

[2] F. Wolf-Monheim, Spectral and Rhythm Features for Audio Classification with Deep Convolutional Neural Networks, Arxiv,Oct. 28, 2024. https://arxiv.org/html/2410.06927v1

[3] E. Lewinson, “Implementing Yann LeCun’s LeNet-5 in PyTorch,” Towards Data Science, Mar. 05, 2025. https://towardsdatascience.com/implementing-yann-lecuns-lenet-5-in-pytorch-5e05a0911320/.

[4] Nouman, “LeNet-5 from Scratch with PyTorch A Beginner’s Guide,” Apr. 22, 2025. https://www.digitalocean.com/community/tutorials/writing-lenet5-from-scratch-in-python#loading-and-transforming-the-data.